2019112602 양정모

Q1 추상 클래스와 인터페이스를 정의하고 구체화 해봅시다

```java
abstract class DonggukStudent {
    String name;
    DonggukStudent(String name) { this.name = name; }
    abstract void study(); // 학생의 역할인 공부
}
```

```java
public interface CatCare {
    void giveChuru(); //고양이 보살피려면 츄르는 필수
}
```

```java
//나의 본질은 동국대생이고, 집사 능력은 추가로 장착함
class Student extends DonggukStudent implements CatCare {
 Student(String name) { super(name); } // 부모 생성자

 @Override
 void study() { System.out.println("공부 중..."); } // 본질 수행

 @Override
 public void giveChuru() { System.out.println("팔정도에 누워있는 동국이에게 츄르를 줍니다."); }
 //추가 능력이지만 필수로 구현되어할 츄르 주기 메소드
}
```

```java
abstract class DonggukStudent {
        String name;
        DonggukStudent(String name) { this.name = name; }
        abstract void study(); //학생의 역할인 공부
}


public interface CatCare {
        void giveChuru(); //고양이 보살피려면 츄르는 필수
}


//나의 본질은 동국대생이고, 집사 능력은 추가로 장착함
class Student extends DonggukStudent implements CatCare {
        Student(String name) { super(name); } //부모 생성자

        @Override
        void study() { System.out.println("공부 중...");} //본질 수행

        @Override
        public void giveChuru() { System.out.println("팔정도에 누워 있는 동국이에게 츄르를
줍니다.");}
        //추가 능력이지만 필수로 구현되어야 할 츄르 주기 메소드
}
```

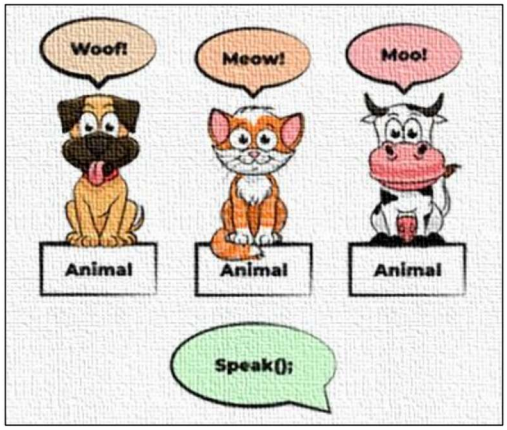
Q2 업 캐스팅과 다운 캐스팅을 활용해 학생 역할과 집사 역할을 제때 수행해봅시다

```java
public class CampusLife {
    public static void main(String[] args) {
        // 1. 업 캐스팅 (Upcasting): 자식 객체를 부모 타입으로 관리
        // 특정 학생을 일단 '동국대 학생'이라는 보편적인 범주로 참조
        DonggukStudent ds = new Student("고윤정");

        System.out.println("=== 1. 업 캐스팅 상태 (학습 모드) ===");
        // 부모(DonggukStudent)가 정의한 본질적인 기능은 바로 사용 가능
        ds.study();

        // ds.giveChuru();
        // 에러 발생 -> 부모 타입 변수로는 자식의 고유 기능(인터페이스 능력)을 참조할 수 없으므로



        System.out.println("\n=== 2. 객체 확인 및 다운 캐스팅 (집사 모드) ===");
        // 2. 다운 캐스팅 (Downcasting): 부모 타입을 다시 자식 타입으로 변환
        // 안전을 위해 instanceof로 먼저 확인
        if (ds instanceof Student) {
            // 명시적 형 변환을 통해 '집사'의 능력을 복원
            Student realStudent = (Student) ds;
            realStudent.giveChuru(); // 이제 고유 기능인 츄르 주기가 가능
        }
    }
}
```

```java
public class CampusLife {
        public static void main(String[] args) {
                //1. 업 캐스팅 : 자식 개체를 부모 타입으로 관리
                //특정 학생을 일단 '동국대 학생'이라는 보편적인 범주로 참조
                DonggukStudent ds = new Student("고윤정");

                System.out.println("=== 1. 업 캐스팅 상태 (학습 모드) ===");
                //부모(DonggukStudent)가 정의한 본질적인 기능은 바로 사용 가능
                ds.study();

                //ds.giveChuru();
                //에러 발생 -> 부모 타입 변수로는 자식의 고유 기능(인터페이스 능력)을 참조할 수
없으므로
                //다운캐스팅을 아래에서 해야한다고 조교님이 설명하셨는데 복습 필요

                System.out.println("\n === 2. 객체 확인 및 다운 캐스팅 (집사 모드) ===");
                //2. 다운 캐스팅 : 부모 타입을 다시 자식 타입으로 변환
                //안전을 위해 instanceof 로 먼저 확인
                if (ds instanceof Student) {
                        //명시적 형 변환을 통해 '집사'의 능력을 복원
                        Student realStudent = (Student) ds;
                        realStudent.giveChuru(); //이제 고유 기능인 츄르 주기가 가능
                        //자식에서 implements 를 해둔 것을 donggukstudent 로 올리는 것이
다운캐스팅
                }
        }
}
```

Q3 지금까지 배운 모든 OOP 역량을 활용해 '다형성'을 구현해봅시다

// 제안 1: 본인만의 역할이 있다면 새로운 인터페이스를 구현해 다중 상속해보세요
// 제안 2: DonggukStudent를 어떠한 기준으로 세분화해서 각자의 공부로 오버라이딩 해보세요




```java
// 컴퓨터공학 전공
class ComputerStudent extends DonggukStudent implements CatCare {
    ComputerStudent(String name) {
        super(name);
    }
    @Override
    void study() {
        System.out.println(name + "이(가) 자바와 알고리즘을 공부 중...");
    }
    @Override
    public void giveChuru() {
        System.out.println(name + "이(가) 고양이에게 츄르를 줍니다.");
    }
}
// 경제학 전공
class EconomicsStudent Sextends DonggukStudent implements CatCare {
    EconomicsStudent(String name) {
        super(name);
    }
    @Override
    void study() {
        System.out.println(name + "이(가) 미시경제와 거시경제를 공부 중...");
    }
    @Override
```

```java
    public void giveChuru() {
        System.out.println(name + "이(가) 고양이에게 츄르를 줍니다.");
    }
}
// 생명과학 전공
class BioStudent extends DonggukStudent implements CatCare {
    BioStudent(String name) {
        super(name);
    }
    @Override
    void study() {
        System.out.println(name + "이(가) 신경과학과 생명과학을 공부 중...");
    }
    @Override
    public void giveChuru() {
        System.out.println(name + "이(가) 고양이에게 츄르를 줍니다.");
    }
}
public class Main {
    public static void main(String[] args) {
        DonggukStudent[] students = new DonggukStudent[3];
        students[0] = new ComputerStudent("정모");
        students[1] = new EconomicsStudent("민수");
        students[2] = new BioStudent("지연");
        for (DonggukStudent s : students) {
            s.study();
        }
        System.out.println("------ 고양이 돌보기 ------");
        for (DonggukStudent s : students) {
            ((CatCare) s).giveChuru();
        }
    }
}
```